

Image Editing Vision Language Model Pipeline Project

Joshua Moffat
San Diego State University
CS 668: Applied Large Language Models

May 12, 2026

Abstract

This project demonstrates how digital photo editing can be executed through language alone; rather than reaching for a brush or a slider in an application like Photoshop, a user describes what they envision in plain English, and the system modifies the image to match. We present a four-phase, natural language-guided image editing pipeline trained on image pairs from the “adjust” edit type of the sysu/ImgEdit dataset. Our initial subset of the ImgEdit dataset contains 10,000 image pairs and after two rounds of filtering 7,818 samples were retained for training. In Phase 1, we fine-tune Qwen2.5-VL-3B-Instruct with QLoRA to predict structured edit instructions including a target region and edit description. In Phase 2, we train a projection adapter and UNet LoRA on top of Stable Diffusion 1.5 inpainting, conditioning the diffusion model on the VLM’s hidden states alongside CLIP text embeddings. In Phase 3, inference chains the fine-tuned VLM through SAM2 mask generation and into the SD inpainting loop. The pipeline is implemented entirely within Google Colab (A100 GPU) with artifacts persisted to Google Drive, introducing constraints around session interruptions, storage limits, and archive streaming throughput that shaped key design decisions. Preliminary testing on the benchmark data set shows that the primary limitation is the quality of the masks produced and the limited training on only one edit type. Easily defined boundaries on objects with more simple shapes are able to be masked properly, though the pipeline fails when trying to edit more complex shapes that have fine details and texture. Phase 4 testing on the benchmark data set has only been developed to perform visual inspection and evaluation of the outputs. Further work can be done to implement another vision language model to judge and score the quality of the outputs.

1 Introduction

Digital photo editing is both an art and a science. Tools like Adobe Photoshop expose every pixel as a manipulable object, an invitation to selectively shape exposure, temperature, and saturation, or to layer entirely separate images into a unified composition. Behind every such edit is a human vision: a photographer translating an emotional intent into a series of precise, practiced operations. The craft is real, but the interface between vision and execution is a steep one. Becoming fluent in that interface demands years of technical training largely orthogonal to the artistic sensibility that motivates the work in the first place.

This paper asks whether natural language can serve as an alternative interface to that same manipulation of light or, more precisely, of pixels. Instead of reaching for a selection tool and a gradient map, a photographer should be able to simply describe what they envision: “make the sky warmer,” “remove the background clutter,” “add more contrast to the subject.” The goal is

not to remove the human from the loop, but to lower the barrier at which human creative intent becomes executable. The learning curve shifts from mastery of a graphical interface to mastery of description — a skill that aligns more naturally with how artists already think about their work.

Realizing this goal requires bridging two of the most powerful paradigms in modern machine learning: vision-language models (VLMs) and generative diffusion models. A VLM can interpret an image alongside a natural-language instruction, identifying which region of the image the instruction refers to and what kind of transformation is implied. A diffusion model can then execute that transformation, sampling pixels that are statistically consistent with both the preserved regions of the source image and the semantic intent encoded in the edit instruction. Together, these two model families form a capable foundation for natural language-driven image editing.

1.1 The Four-Phase Pipeline

Our system is organized into four sequential phases, each implemented as a self-contained Google Colab notebook:

Phase 0 performs dataset acquisition and quality auditing. We download a subset of the ImgEdit dataset (Ye et al. 2025) from HuggingFace (*'sysuuy/ImgEdit'*), pairing edit instruction prompts from Parquet manifests with source and target images extracted from tar-split archives. We additionally join ground-truth RLE segmentation masks from the companion *'sysuuy/ImgEdit_recap_mask'* dataset to provide supervision for spatial editing tasks.

Phase 1 fine-tunes the Qwen2.5-VL-3B-Instruct model using QLoRA and the TRL 'SFT-Trainer' framework to predict structured JSON edit outputs from paired (image, instruction) inputs. After fine-tuning, we extract and cache VLM hidden states for all training samples to Google Drive.

Phase 2 trains the diffusion bridge: a VLMProjectionAdapter and a set of UNet LoRA adapters are trained jointly using the pre-cached VLM hidden states and ground-truth RLE masks. The VAE, CLIP text encoder, and UNet base weights are all frozen throughout this phase. Training uses the standard epsilon-prediction denoising diffusion probabilistic models (DDPM) objective with mixed-precision forward passes and gradient accumulation.

Phase 3 assembles the full inference pipeline, chaining five components in sequence. An input image and natural-language instruction are first passed through (1) the fine-tuned Qwen2.5-VL-3B VLM, which produces a structured JSON prediction containing the `edit_type`, `edit_description`, and a subject phrase, while simultaneously extracting the hidden-state conditioning vector used to drive the diffusion model. The subject phrase is then passed to (2) GroundingDINO (Liu et al. 2024), a zero-shot open-vocabulary detector (IDEA-Research/grounding-dino-tiny), which localises the target region in the image and returns a precise bounding box. This replaces the earlier design in which the VLM was asked to predict bounding-box coordinates directly; that approach produced degenerate boxes on a substantial fraction of held-out samples. The GroundingDINO bounding box is then fed to (3) SAM2 (Ravi et al. 2024), Meta's Segment Anything Model 2, which converts it into a binary segmentation mask. Finally, (4) the trained Stable Diffusion 1.5 inpainting UNet, conditioned on the CLIP-plus-projected-VLM combined embedding, generates the edited image within the masked region over a DDIM denoising loop with classifier-free guidance.

Phase 4 evaluates the Phase 3 pipeline against the singleturn split of the ImgEdit benchmark (Ye et al. 2025), comprising 737 images spanning 9 edit-type categories (action, add, adjust, background, compose, extract, remove, replace, and style). Of the 737 images we chose to test only the edits from the 'adjust' edit type, since that was the same type of edit the models were trained

on, which gave us 67 images to test. The primary evaluation is visual inspection of the output. In the future, a VLM image edit judge could be implemented, in which it would score each pipeline output on four criteria—*edit correctness*, *edit quality*, *localisation accuracy*, and *preservation of unedited regions*—each on a 1–5 scale, using per-edit-type rubrics supplied by the benchmark’s `judge_prompt.json`.

The entire pipeline runs on Google Colab Pro using an NVIDIA A100 GPU (40 GB VRAM). The choice of cloud execution introduces a set of non-trivial engineering constraints that shape nearly every design decision in the system. Colab sessions terminate after a fixed wall-clock duration, requiring all expensive intermediate artifacts (dataset samples, VLM hidden states, model checkpoints) to be persisted to Google Drive after each computation. Drive’s upload and download quotas and the overhead of streaming large files across the Drive API necessitated conservative choices about dataset size and sharding strategy.

The remainder of this paper is organized as follows: Section 2 describes a VLM and diffusion model; Section 3 presents the dataset, data engineering choices, the full system architecture and training methodology for each phase; Section 4 reports qualitative inference results; Section 5 discusses constraints, known issues, and directions for future work.

2 Background

2.1 Vision-Language Models

Vision-language models extend the transformer architecture to accept both visual and textual inputs, enabling a single model to reason jointly about image content and natural language. In this work we use Qwen2.5-VL-3B-Instruct (Wang et al. 2024), a 3-billion-parameter multimodal transformer from Alibaba’s Qwen team. The model accepts interleaved image patches and text tokens, processes them through a shared transformer backbone, and produces autoregressive text output. For our purposes, we fine-tune this model to produce structured JSON predictions that identify the edit type, a bounding box identifying the target region (expressed in normalized $[0, 1000]$ coordinates), and a refined edit description. We fine-tune using QLoRA (Quantized Low-Rank Adaptation) (Dettmers et al. 2023), attaching trainable low-rank delta matrices to the language-model attention projections while keeping both the frozen base weights and the vision encoder in 4-bit NF4 quantization. This reduces GPU memory requirements substantially while preserving the expressive capacity needed for spatial reasoning.

Beyond generating text predictions, the VLM also serves as a conditioning signal for the downstream diffusion model. Specifically, we extract the mean-pooled hidden state from the model’s final transformer layer over the user-turn text tokens, excluding image patch tokens and special chat-template tokens, to obtain a 2048-dimensional embedding that captures the semantic intent of the edit instruction as understood by the fine-tuned VLM. These hidden states are pre-computed and cached to persistent storage, decoupling the VLM from the diffusion training loop entirely.

2.2 Diffusion Models for Image Inpainting

Denoising diffusion probabilistic models (DDPMs) (Ho, Jain, and Abbeel 2020) learn to reverse a gradual noising process, generating samples by iteratively denoising a randomly initialized latent representation. Latent diffusion models (LDMs) (Rombach et al. 2022) apply this process in a compressed latent space encoded by a variational autoencoder (VAE), dramatically reducing

computational cost. The Stable Diffusion 1.5 inpainting variant (Rombach et al. 2022) extends this architecture with a 9-channel UNet input: the 4-channel noisy latent is concatenated with a single-channel inpaint mask and the 4-channel VAE encoding of the source image masked to zero in the target region. This design preserves source-image context for unmasked regions while allowing unconstrained generation within the mask, making it well-suited to localized edit tasks.

We condition the inpainting UNet with a combined 78-token sequence composed of a 77-token CLIP text encoding of the edit instruction concatenated with a single projected VLM token. The VLM token is produced by a lightweight 2-layer MLP projection adapter, trained jointly with UNet LoRA adapters, that maps the 2048-dimensional VLM hidden state into the 768-dimensional cross-attention space of the UNet. This architecture allows the diffusion model to leverage both the broad semantic grounding of CLIP and the instruction-specific spatial reasoning encoded in the fine-tuned VLM’s hidden states.

3 Methods

System Overview

This work implements a four-phase multimodal image-editing pipeline that chains a fine-tuned vision-language model (VLM) with a Stable Diffusion 1.5 inpainting model. Given a source image and a natural-language edit instruction, the system identifies the region to be edited and renders the requested change into that region. The pipeline is divided into four phases executed sequentially: (0) dataset acquisition and quality audit, (1) VLM fine-tuning for structured edit prediction, (2) Stable Diffusion bridge training that learns to condition on VLM representations, and (3) an inference chain that routes a new image through the trained components to produce an edited output. Phase 4 applies the inference pipeline to a held-out benchmark subset and evaluates outputs through visual inspection.

Dataset

Training data is drawn from the ImgEdit dataset (*‘sysuuy/ImgEdit’*) hosted on HuggingFace. ImgEdit is a large-scale image editing dataset organized into 10 single-turn edit types: action, add, adjust, background, content, hybrid, reference, remove, replace, and style. The dataset stores source and target image paths in Parquet files while the images themselves reside in *‘.tar.split’* archives of approximately 42.9 GB each. Segmentation masks are provided in a separate companion dataset (*‘sysuuy/ImgEdit_recap_mask’*), which contains per-sample JSON annotations stored inside tar archives under a *‘jsons/’* directory prefix. Each annotation provides a bounding box in absolute pixel coordinates (*‘xyxy’* format) and a COCO run-length-encoded (RLE) mask string for every foreground and background region detected by a SAM2-based annotator.

Because the main dataset and the mask dataset are hosted separately, a join is required at pre-processing time. The join key is the sample ID extracted from the image file path (the second path component under the archive root, e.g., *‘results_adjust_laion_part0/00121_00035_000359496/original.png’* to *‘00121_00035_000359496’*). During Phase 0, mask annotation JSONs are downloaded from *‘jsons/part0.tar’* through the HuggingFace Hub API and indexed by sample ID. Google Drive rate limit warnings were encountered during the download of the annotation JSON files and images to Drive; however, the download completed before the limit was reached.

Sampling Strategy and Data Constraints

The ImgEdit dataset’s image archive structure presented particular challenges. Images are distributed across ‘tar.split’ files of approximately 42.9 GB each, hosted on HuggingFace. A stratified random sample across edit types would require searching through multiple split files in an unpredictable order, a “needle in a haystack” problem that proved impractically slow in the Colab streaming environment. We therefore restrict our training corpus to the “adjust” edit type, from which the first 10,000 image pairs could be extracted sequentially from the first split file without multi-archive traversal. After applying data quality filters (requiring a valid joined RLE mask, a non-degenerate bounding box annotation, and samples below the max tokenized length) the effective training set consists of a subset of 7,818 of the 10,000 samples.

Training duration further constrained our hyperparameter choices. An initial pilot with 1 epoch of VLM fine-tuning and 2 epochs of diffusion training produced a visibly undertrained model. The current training configuration uses 3 epochs for Phase 1 and 9 epochs for Phase 2. Further training is likely needed to produce quality outputs for complex edits, though for this illustration the pipeline is able to modify objects with simple shapes, like changing the color of a T-shirt.

Phase 1: Vision-Language Model Fine-Tuning

Phase 1 fine-tunes ‘*Qwen/Qwen2.5 – VL – 3B – Instruct*’, a 3-billion-parameter multimodal language model with a native vision encoder, for the task of structured edit prediction. The model is selected as a balance between capability and the memory constraints of the Colab A100 runtime, which provides 40 GB of VRAM. Loading the full model in 32-bit precision at this scale could exceed VRAM; accordingly, the model is loaded with 4-bit NF4 quantization via ‘bitsandbytes’, using BF16 as the compute dtype. This reduces the effective memory footprint of the frozen base weights, leaving headroom for activations, LoRA parameters, and the optimizer. This pipeline is intended as an illustration of the approach. In practice, higher-end hardware or distributed training would permit the use of a larger parameter model, which would likely produce stronger edit predictions and higher-quality hidden-state representations for Phase 2 conditioning.

Parameter-efficient fine-tuning is applied via Low-Rank Adaptation (LoRA) using the PEFT library. Adapters are attached exclusively to the attention projection layers of the language model transformer, specifically the query, key, value, and output projections (‘q_proj’, ‘k_proj’, ‘v_proj’, ‘o_proj’). The vision encoder’s fused QKV projection (‘qkv’) is deliberately excluded; training the vision encoder would risk distorting the general visual representations the model acquired during pretraining. The LoRA rank is set to 16 with a scaling factor (alpha) of 32 and dropout of 0.05. This yields approximately 20–25 million additional trainable parameters against a frozen 3-billion-parameter base.

The model is trained to predict, from a source image and a natural-language edit instruction, a structured JSON object of the form:

```
{
  "edit_type": "adjust",
  "bbox": [x1, y1, x2, y2],
  "edit_description": "...",
}
```

Bounding box coordinates are expressed in a ‘[0, 1000]’ relative scale (0 = top-left corner, 1000 = bottom-right corner). This coordinate system is used consistently in both the context provided to

the model and the supervision target, so the model can learn to copy the correct region coordinates directly from the scene description without requiring an implicit scale conversion. The user-turn prompt presents the source image alongside a list of all segmentation regions detected in that image. Including all regions — rather than pre-selecting a single candidate — requires the model to learn the spatial reasoning task of associating the edit instruction with the correct object. Ground-truth region selection for the supervision target is performed via class-name matching against the edit instruction, with CLIP score and region area as tiebreakers. A fixed system prompt and prompt template are shared across training data generation, hidden-state extraction, and inference. Strict template consistency is enforced to prevent training/inference distribution mismatch; the template is hashed and the hash is embedded in saved checkpoints as a guard.

The model is trained using SFTTrainer from the TRL library with a pre-tokenized `'torch.utils.data.Dataset'` (rather than a formatting function), which is necessary to prevent the trainer from silently dropping multimodal fields such as `'pixel_values'`. The `'remove_unused_columns=False'` flag is required to prevent the Trainer from dropping inputs that are passed through `'kwargs'` in Qwen2.5-VL's forward signature. Key hyperparameters are: 3 training epochs, a per-device batch size of 2 with gradient accumulation of 8 (effective batch size 16), a learning rate of 2×10^{-4} with a cosine schedule and 5% warmup ratio, and a maximum sequence length of 2,560 tokens. The optimizer is `'paged_adamw_8bit'`, which reduces optimizer state memory by approximately 75% compared to standard AdamW.

After fine-tuning, the trained VLM is used to extract a fixed-dimensional conditioning vector for each training sample, which will be consumed by the Phase 2 diffusion model. For each sample, a user-turn-only forward pass is performed with `'output_hidden_states=True'`. The last hidden layer produces a tensor of shape `'(1, seq_len, 2048)'`. Before mean pooling, tokens with IDs greater than or equal to 151,643 are excluded; this threshold separates regular vocabulary tokens from Qwen2.5-VL special tokens, including image patch tokens (`'—image_pad—'`) and chat-template delimiters (`'—im_start—'`, `'—im_end—'`). Mean pooling over the retained text tokens yields a single (2048) float32 vector per sample.

Pre-caching these vectors to Google Drive (rather than computing them on-the-fly during Phase 2 training) is critical for two reasons. First, it eliminates the need to keep the 3B VLM loaded simultaneously with the diffusion UNet during training, which could exceed available VRAM. Second, it reduces the per-step training overhead to a negligible Drive read of a precomputed vector. Vectors are stored in sharded `'.pt'` files indexed by a manifest that records the shard file and row position for each sample.

Phase 2: Stable Diffusion Inpainting Bridge Training

Phase 2 trains a bridge between VLM representations and a Stable Diffusion 1.5 inpainting model (*runwayml/stable-diffusion-inpainting*). The SD 1.5 inpainting variant uses a modified UNet with 9 input channels rather than the standard 4. The 9 channels carry: 4 channels of the noisy target latent, 1 channel of the downsampled binary mask, and 4 channels of the masked source image latent (i.e., the source image with the edit region zeroed out). This 9-channel architecture is the standard inpainting interface for SD 1.5 and allows the model to attend to unmasked source content when generating edits.

The pre-cached VLM hidden states occupy a 2,048-dimensional space, while the UNet's cross-attention mechanism expects conditioning vectors in a 768-dimensional space (matching the CLIP text encoder output). A learnable `'VLMProjectionAdapter'` maps between these spaces. Its ar-

chitecture is a two-layer MLP: ‘Linear(2048 $\rightarrow$ 1536) $\rightarrow$ LayerNorm $\rightarrow$ GELU $\rightarrow$ Linear(1536 $\rightarrow$ 768)’. The intermediate dimension of 1,536 (twice the target dimension) provides sufficient expressive capacity to bridge the gap between the VLM and CLIP representation spaces, which differ in training objective, vocabulary, and architecture (without overfitting at the available dataset scale). The output has shape (batch, 1, 768), representing a single new cross-attention token.

At each training step, the VLM projection token is concatenated along the sequence dimension with the CLIP text embedding of the edit instruction. The CLIP encoder produces a tensor of shape (batch, 77, 768) from a maximum-length tokenization of the instruction; concatenating the VLM token produces a combined conditioning tensor of shape (batch, 78, 768). This combined tensor is passed as ‘encoder_hidden_states’ to the UNet. The concatenation approach, rather than addition, allows the UNet’s cross-attention to learn independently when to attend to CLIP-derived versus VLM-derived information, and avoids requiring both streams to be the same shape.

During inference, classifier-free guidance (CFG) is applied. The unconditional branch uses a CLIP embedding of the empty string and a zero tensor in place of the VLM token. The UNet receives a batch of size 2 (unconditional concatenated with conditional), and the final noise prediction is computed as: *‘noise_pred = uncond + guidance_scale * (cond - uncond)’*.

Rather than fine-tuning the full UNet (approximately 860M parameters), LoRA adapters are attached to the query, key, value, and output projections of all attention layers, both self-attention and cross-attention. Adapting cross-attention layers is particularly important because cross-attention is where the conditioning tensor interacts with image features; the base SD 1.5 model was trained with 77-token CLIP-only conditioning, and adapting these layers teaches it to incorporate the new 78-token conditioning structure that includes the VLM token. The LoRA rank for Phase 2 is 16 (alpha 32), having been increased from an initial rank of 8 after the lower-capacity configuration produced blurry or color-flattened outputs on attribute-transformation edits such as hair color changes. The training objective is standard epsilon-prediction MSE: noise is sampled from a DDPM scheduler and added to the target image latent at a random timestep, and the UNet is trained to predict the added noise. Training uses mixed-precision (fp16 autocast) with a GradScaler to prevent underflow, a per-device batch size of 1 with gradient accumulation of 16 (effective batch size 16), a learning rate of $1 * 10^{-4}$, and a linear warmup of 200 steps followed by cosine decay over 9 epochs. Training completed in approximately 2 hours on an A100 GPU.

A critical design element is that Phase 2 training and Phase 3 inference use distinct mask sources that are never mixed. During training, binary inpainting masks are derived from the ground-truth RLE annotations in the companion mask dataset. During inference, masks are generated by the SAM2 segmentation model using bounding boxes from GroundingDINO (described in Phase 3). This separation ensures that the diffusion model’s training objective is always conditioned on precise, annotated masks, while inference uses automatically predicted masks without contaminating the training signal.

Phase 3: Inference Pipeline

At inference time, the pipeline chains four models: the fine-tuned VLM, GroundingDINO, SAM2, and the trained SD inpainting model. Each component runs sequentially on the source image and edit instruction.

The fine-tuned Qwen2.5-VL model is loaded with its Phase 1 LoRA adapter and runs in evaluation mode. Given the source image and instruction, it generates a structured JSON prediction containing the predicted ‘edit_type’, a bounding box ‘[x1, y1, x2, y2]’ in ‘[0, 1000]’ coordinates, and

an ‘edit_description’ that describes what specifically should be changed. Simultaneously, a separate user-turn-only forward pass (without generation) extracts the VLM hidden state, applying the same token filtering and mean-pooling procedure used during Phase 1 to produce the ‘(2048)’ conditioning vector. A known distribution gap exists between training and inference for hidden-state extraction: during training, the user prompt included a full list of segmentation region annotations, whereas at inference time no ground-truth annotations are available and the prompt contains only the image and instruction. Both VLM generation and hidden-state extraction at inference use the same annotation-free prompt, so they are internally consistent with each other; the gap is between the training-time and inference-time prompt distributions.

Early versions of the inference pipeline fed the VLM-predicted bounding box directly to SAM2 for mask generation. This approach produced two systematic failure modes: degenerate VLM bounding boxes (e.g., ‘[0, 0, 1000, 1000]’ covering the entire image) caused SAM2 to generate unfocused masks fragmented across the image, and under-localized boxes caused SAM2 to segment only a partial object. In response, Phase 3 v3.0 introduces ‘*IDEA – Research/grounding – dino – tiny*’ as an intermediate localization step.

GroundingDINO is a pretrained open-vocabulary object detector that accepts a free-form text phrase and returns calibrated bounding boxes with confidence scores. Rather than using the full VLM-generated JSON output, a rule-based function extracts a concise subject noun phrase from the ‘edit_description’ field (e.g., “the motorcycle”, “the bread on the plate”). GroundingDINO is queried with this phrase and returns candidate boxes. The highest-confidence box is used as the SAM2 prompt when available; the system falls back to the VLM-predicted box only when GroundingDINO produces no detections. For edits targeting the global image (identified by keywords such as “brightness,” “contrast,” or “saturation”), SAM2 is bypassed entirely and a full-image mask is used. The HuggingFace Transformers port of GroundingDINO is used in preference to the standalone ‘groundingdino-py’ package, which requires compiling CUDA extensions that are incompatible with some Colab GPU environments. A known dtype mismatch in the Transformers port’s text branch requires loading the model in full float32 precision rather than float16.

SAM2 (‘SAM2.1 Hiera Large’, checkpoint ‘sam2.1_hiera_large.pt’) generates the inpainting mask from the localization bounding box. The SAM2ImagePredictor API is used with ‘multi_mask_output=True’, which returns up to three candidate masks per prompt. Candidate masks are reranked by CLIP image-text similarity to the subject phrase extracted in the previous step; the highest-ranked mask is selected. Post-processing applies morphological closing to smooth jagged edges, followed by selection of the largest connected component to eliminate detached mask fragments. The quality of the generated mask is the most consequential factor in final output quality. A precise mask that tightly follows the object boundary allows the inpainting model to fill only the intended region while preserving surrounding content. Imprecise masks (whether too large, too small, or incorrectly shaped) limit the inpainting model’s ability to produce a coherent edit regardless of how well the diffusion model was trained. SAM2 is not distributed on PyPI and must be installed from the GitHub source repository (‘facebookresearch/sam2’) at each Colab session. The SAM2 checkpoint is cached to Google Drive to avoid re-downloading the 1.2 GB file on each restart. A known incompatibility between SAM2’s Hydra-based configuration system and Python 3.12 editable installs requires pre-initializing Hydra with an absolute filesystem path to the SAM2 config directory before loading the model.

The final step applies the trained SD 1.5 inpainting model to the source image using the SAM2-generated mask as the edit region. The DDIM scheduler is used for inference with 20 denoising

steps (replacing the DDPM training scheduler). Combined conditioning is assembled identically to Phase 2 training (CLIP encoding of the instruction concatenated with the VLM projection token) but with CFG enabled (`'do_cfg=True'`), which doubles the batch dimension to accommodate the unconditional branch. All images are processed at 512 x 512 resolution, which is the native resolution of SD 1.5.

Phase 4: Benchmark Evaluation

Evaluation is performed against the 67 ‘adjust’-type samples from the ImgEdit singleturn benchmark. These samples represent image pairs and edit instructions that were not present in the training set, allowing assessment of the pipeline’s generalization to unseen inputs. The benchmark archive (`'Benchmark.tar'`) is downloaded from the HuggingFace repository and parsed to extract sample metadata including source image paths, reference target images, and edit instructions. For each benchmark sample, the full Phase 3 inference pipeline is applied to produce an edited image. Due to time constraints, evaluation is conducted through direct visual comparison of the pipeline output against the reference target image rather than through automated metrics and VLM judge. Observations from this evaluation indicate that the pipeline performs better on edits targeting large objects with well-defined, geometrically simple boundaries, such as changing the color of a dress or a car, where GroundingDINO and SAM2 reliably produce tight, accurate masks. Performance degrades on edits targeting objects with complex or irregular boundaries, such as animals, buildings, and food, where mask edges are less precisely defined and the inpainting model must work with a noisier region specification.

Software Environment

Key software dependencies and versions are listed in Table 1. Version pins were chosen to ensure compatibility with Qwen2.5-VL’s native Transformers integration, PEFT’s LoRA API stability, and SAM2’s PyTorch requirement.

4 Results

Due to the time constraints of this project, evaluation using a VLM judge to score the quality and accuracy of the generated edited images against a held-out test set was not feasible. The singleturn split of the ImgEdit benchmark (Ye et al. 2025) does not supply ground-truth edited images, precluding the use of pixel-level similarity metrics such as SSIM or LPIPS. Accordingly, our evaluation consists of *visual inspection* of the pipeline outputs on 67 previously unseen images drawn from the adjust edit-type category of the benchmark. This subset was chosen because the models were also only trained on adjust edits and represent a manageable scope for the model’s current capacity. For each sample, we examine three outputs produced by the pipeline: the source image, the localization mask generated by the GroundingDINO (Liu et al. 2024) – SAM2 (Ravi et al. 2024) sub-pipeline, and the final inpainted output from the fine-tuned Stable Diffusion 1.5 inpainting model (Rombach et al. 2022).

The goal is not to demonstrate state-of-the-art performance but to validate that all components of the end-to-end pipeline function correctly, to characterize the classes of edits the system handles well versus poorly, and to identify the dominant failure modes that future work should address.

Pipeline Summary Statistics

Table 2 summarizes the pipeline’s behavior across all 67 benchmark samples. Every sample completed inference without a runtime error, giving a 100% completion rate. In 57 of 67 cases (85.1%), the localization sub-pipeline successfully detected the target object using GroundingDINO and passed the resulting bounding box to SAM2. The remaining 10 samples (14.9%) triggered the global-edit fallback, which masks the entire image, because GroundingDINO found no credible detections. The mean per-sample inference time is approximately 8.4 seconds on an NVIDIA A100 40 GB GPU. The wide gap between mean mask coverage (31.0%) and median mask coverage (17.8%) indicates that the distribution is right-skewed: most masks are compact and object-sized, but a minority of global-edit fallbacks inflate the mean. The GroundingDINO confidence scores cluster around 0.6, which is consistent with the model operating on object-level noun phrases extracted from natural-language edit instructions rather than object class names. The confidence score is interpreted as the vision language alignment confidence between the bounding box and the text prompt. We would expect higher confidence scores if we used only object class names, but the target edit does not always apply to all objects of that class. For example, an image may contain multiple clothing items, so we chose to use object-level phrases for greater specificity of which clothing item to edit. Lower confidence scores were typically observed for abstract or compound subjects where the grounding model has less direct training signal.

Package	Version constraint	Ph. 0	Ph. 1	Ph. 2	Ph. 3	Ph. 4
torch	$\geq 2.5.1$	✓	✓	✓	✓	✓
torchvision	(bundled with torch)			✓		
numpy	< 2	✓	✓	✓	✓	✓
Pillow	≥ 10.0	✓	✓	✓	✓	✓
transformers	$= 4.49.0$	✓	✓	✓	✓	✓
diffusers	$= 0.30.0$	✓		✓	✓	✓
peft	$= 0.12.0$		✓	✓	✓	✓
accelerate	$= 0.34.0$		✓	✓		
trl	$= 0.11.0$		✓			
bitsandbytes	$= 0.45.4$ / $\geq 0.44.0$		✓		✓	✓
qwen-vl-utils	$\geq 0.0.8$		✓		✓	✓
sam2	GitHub main	✓			✓	✓
hydra-core	$\geq 1.3.2$				✓	✓
torchao	$\geq 0.16.0$			✓	✓	✓
datasets	$= 2.21.0$	✓				
huggingface-hub	(transitive dep.)	✓				✓
pycocotools	$\geq 2.0.7$	✓	✓	✓	✓	✓
pyarrow	(transitive dep.)	✓				
pandas	(transitive dep.)	✓				
tqdm	≥ 4.66	✓	✓	✓		
packaging	(transitive dep.)	✓	✓			✓
scipy	(transitive dep.)				✓	
lpips	(no pin)					✓
scikit-image	$\geq 0.21.0$					✓

Table 1: Required Python packages and version constraints by pipeline phase. Ph. 0 = dataset setup & audit; Ph. 1 = VLM QLoRA fine-tuning; Ph. 2 = SD1.5 bridge training; Ph. 3 = inference pipeline; Ph. 4 = benchmark & evaluation. Phases 0–1 use exact pins ($=$); Phases 2–4 use lower-bound pins ($\geq$).

Table 2: Benchmark inference statistics across 67 `adjust`-type samples.

Metric	Value
Total benchmark samples	67
Successful inference runs	67 (100%)
GroundingDINO localization used	57 (85.1%)
Global-edit fallback (no detection)	10 (14.9%)
Mean GroundingDINO confidence score	0.608
Median GroundingDINO confidence score	0.579
Mean mask coverage (fraction of image)	31.0%
Median mask coverage (fraction of image)	17.8%
Mean inference time per sample	8.4 s
Phase 1 VLM training samples	7,818
Phase 1 final training loss	0.114
Phase 2 SD bridge initial training loss	0.080
Phase 2 SD bridge final training loss	0.030

Successful Edits

The pipeline performs most reliably on edits involving a single, visually distinct, uniformly colored object with smooth contours. Figure 1 illustrates this with sample `st_1058`, which requested that the color of a dress be changed to soft pink. The localization sub-pipeline produced a near-complete mask covering the dress with only a minor omission near the hem. The diffusion model correctly interpreted the target color from the edit instruction and produced a soft pink dress with visible pleating, demonstrating that the fine-tuned UNet is capable of generating fine structural detail within the continuous inpainted region. The high contrast between the blue dress and the background made the grounding and masking steps straightforward, and the smoothly bounded shape of the garment meant the diffusion model could fill the region continuously rather than navigating complex occlusions or fine structure. This example illustrates the conditions under which the pipeline achieves its best output quality: an unambiguously localizable subject, a clean binary mask, and a color attribute edit that does not require the model to synthesize novel textures or object geometry.

Partial Successes

The majority of benchmark outputs fall into the partial-success category. The pipeline correctly identifies the target object, generates a plausible mask, and applies the correct color; however, the output contains visible artifacts, boundary imprecision, or localized degradation. The dominant failure pattern at this tier is mask leakage into adjacent regions, which causes the diffusion model to repaint areas that should be preserved.

Garment with facial boundary leakage (st_1048). Figure 2 shows the result for the instruction “*Change the priest’s coat color to dark brown.*” The pipeline generated a suitable mask for the coat, but the upper boundary of the mask extends into the lower portion of the face. As a consequence, the inpainted output correctly applies a dark brown coat, but the region near the chin and jaw exhibits inconsistent color and reduced detail compared to the original. This is a representative case of mask boundary imprecision on objects that share a border with a human face, where SAM2 must distinguish between the garment and skin at close range.

Vehicle with paint bleed and rough detail (st_1090, st_1091, st_1092). Transportation subjects—boats, motorcycles, and cars—formed a large share of the benchmark and consistently fell into the partial-success tier. These objects pose two compounding challenges: their complex silhouettes require precise masks across wheels, windows, and chassis protrusions, and the diffusion model must simultaneously apply a new color *and* reconstruct the fine structure of the repainted surface.

Figure 3 shows three representative vehicle edits. For st_1090 (“*Change the yacht’s hull color to navy blue*”), the pipeline produced a sufficient mask and correctly painted the hull, but also repainted the superstructure of the boat, suggesting the mask included more of the vessel than intended. The resulting image contains a correctly shaped boat with visible windows and railings, but with noticeable artifacting and less detail than the original. For st_1091 (“*Change the color of the motorcycle to matte black*”), the mask covered the bike body well but incorrectly filled several gaps between the wheel spokes. The diffusion model applied matte black convincingly to the frame, but the handlebars and seat geometry are deformed, and the rear of the bike shows greenish color contamination from the background foliage. For st_1092 (“*Change the color of the vehicle to blue*”), the mask included the front wheel incorrectly while omitting the rear, and contained small holes through which the original red paint is visible. The diffusion model correctly identified the target color as blue but was unable to repaint the details of the race car, like the racing decals and designs.

Watercraft with structural complexity (st_1098, st_1099). Figures 4 and 5 show two further watercraft edits. For st_1098 (“*Change the luxury boat color to red*”), the mask correctly isolated the boat while preserving the people aboard, and the diffusion model applied a recognizable red speedboat shape. The interior rear of the boat appears deformed, but the overall composition is close enough to the original to be recognizable. For st_1099 (“*Change the seaplane’s color to red*”), the pipeline masked the fuselage and wings but failed on the engine and propeller assembly, which have far more intricate geometry. The resulting output correctly repaints the large planar surfaces red but leaves the propeller area inconsistent—an example where even a broadly correct mask quality is insufficient when the edit region contains high-frequency structural detail that the diffusion model cannot reconstruct at the resolution and training scale used.

Failure Cases

Several samples expose categorical limitations of the pipeline that simple improvements to the localization or diffusion steps are unlikely to resolve without more training data and a more diverse sample. These are presented in Figure 6.

Animal fur: complete subject repaint (st_1073). The instruction for st_1073 was “*Change the animal’s fur color to a solid shade of brown.*” The pipeline produced a plausible mask covering the body of the animal. However, because the fur constitutes the entire surface of the subject, the diffusion model was required to repaint the complete visible object, functionally equivalent to synthesizing a new animal from scratch within the masked region. Stable Diffusion 1.5 without fine-tuning is known to struggle with realistic animal generation, and the fine-tuning data at this scale did not provide sufficient examples to correct for this. The output replaced the bear with an object that resembles a large, round cat rather than the original species, with reddish-colored artifacts around the boundary. This failure illustrates a fundamental constraint of the inpainting approach: when the mask encompasses the whole object, the model must generate novel content rather than simply recoloring, which exposes the base model’s limitations in realism and that the fine-tuning data sample has limited diversity/quantity in animal examples.

Architectural facade: over-complex mask subject (st_1106). For st_1106, the instruction was “*Change the building’s facade color to light grey.*” A building facade is not a single, segmentable object; it is a structured surface that must be distinguished from windows, doors, and decorative elements, none of which should be recolored. The pipeline could not produce an appropriate mask: the output mask was jagged, contained holes, omitted part of the facade near ground level, and failed to exclude the window regions. The diffusion model, unable to reconstruct the facade structure within a poor mask, produced a flat grey fill over the masked area with no structural detail. This case reveals that the pipeline can segment whole objects fairly well but not fine-grained components of architectural structures and the fine-tuning data sample lacks diversity in architectural examples due to its failure to produce any details resembling the original structure.

Conflicting color instruction (st_1130). The instruction “*Change the surface color of the white baby onesie to red*” contains an inherent ambiguity: the modifier “white” specifies the *current* color of the surface, while “red” specifies the target. The pipeline generated a valid mask covering the onesie, but the diffusion model failed to follow the instruction, painting the onesie white rather than red. The VLM may not have trained on sufficient examples of edits where the current color of an object is named explicitly in the instruction, leading the model to anchor on the word “white” as the target state rather than the source state. This represents a semantic parsing failure within the diffusion conditioning rather than a localization problem.

Ambiguous subject with no object anchor (st_1135). For st_1135, the instruction was “*Change the frosting color to a light blue.*” The instruction does not name the food item bearing the frosting, providing no anchor noun for GroundingDINO to localize. The grounding model returned a single detection, and SAM2 produced a mask, but the resulting mask was inadequate—failing to coherently capture the frosting surface while leaving the underlying food structure exposed. The diffusion model, receiving an ambiguous masked region and instruction, hallucinated cupcakes with unrealistic light blue frosting, generating an output that bears little structural resemblance to the source image and contains numerous holes where original content bleeds through. This case demonstrates two compounding weaknesses: the localization sub-pipeline requires a clear nominal subject, and the diffusion model has insufficient exposure to complex food textures to generate plausible results even when localization partially succeeds.

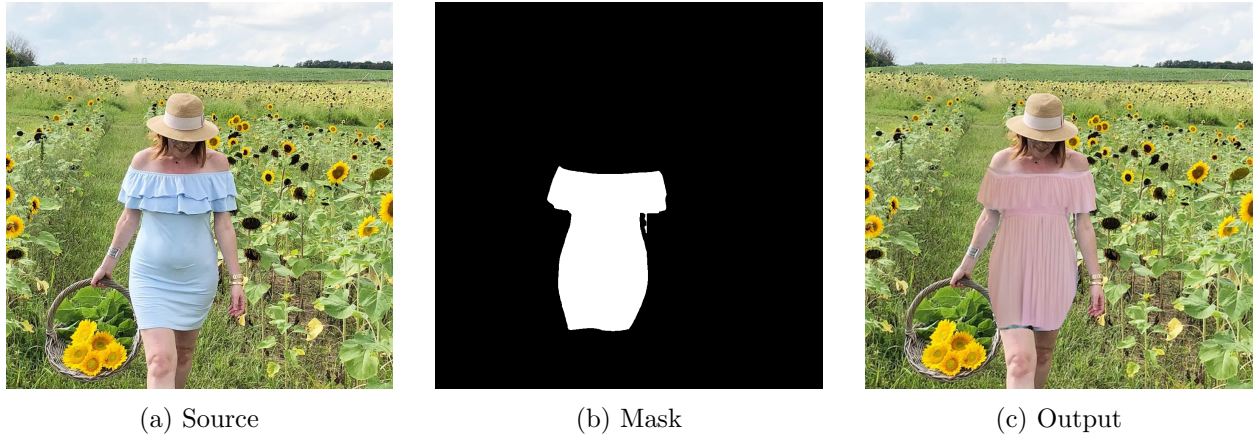


Figure 1: **st_1058**: “*Change the color of the dress to a soft pink.*” The pipeline produces a near-complete mask and correctly applies a soft pink color with visible pleating in the garment. This represents the pipeline performing at its best: a visually distinct, smoothly bounded subject with a simple color-change instruction.

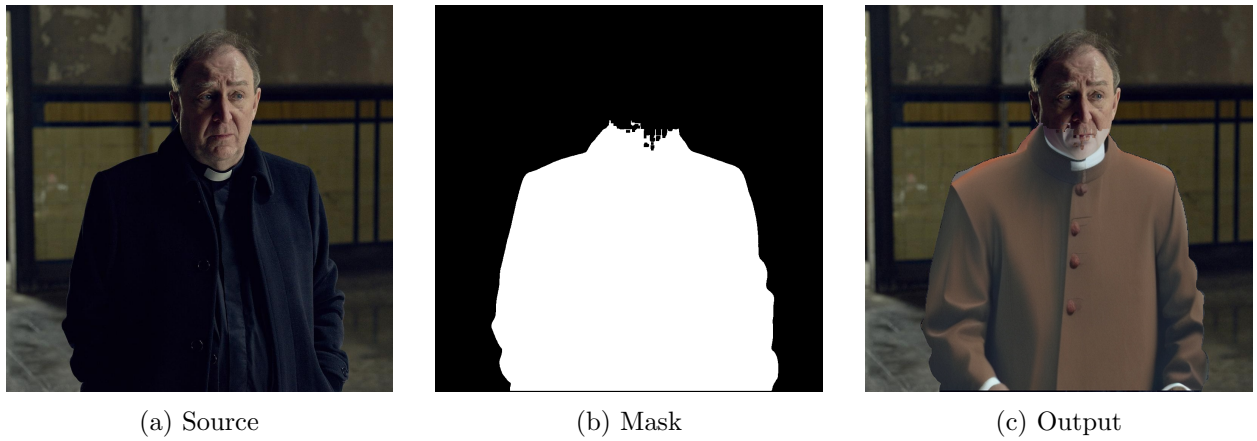
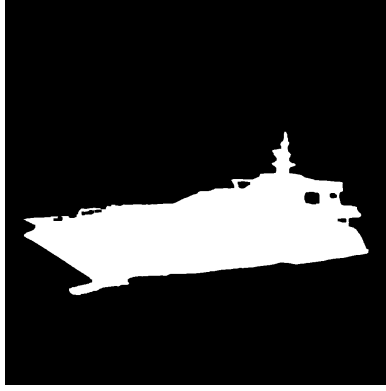


Figure 2: **st_1048**: “*Change the priest’s coat color to dark brown.*” The mask leaks into the lower face, causing loss of facial detail in the output despite the coat color being changed correctly.



(a) st_1090 Source



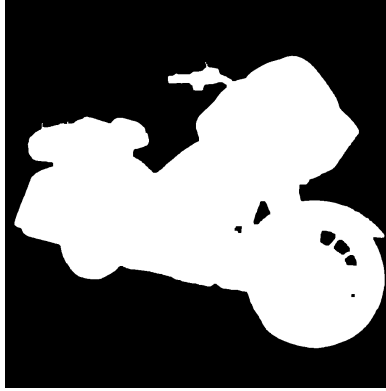
(b) st_1090 Mask



(c) st_1090 Output



(d) st_1091 Source



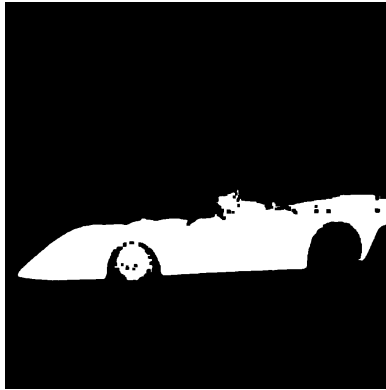
(e) st_1091 Mask



(f) st_1091 Output



(g) st_1092 Source



(h) st_1092 Mask



(i) st_1092 Output

Figure 3: Vehicle edit partial successes. **st_1090**: “Change the yacht’s hull color to navy blue” - correct color but paint extends to superstructure. **st_1091**: “Change the color of the motorcycle to matte black” - correct color with deformed geometry and background contamination. **st_1092**: “Change the color of the vehicle to blue” - correct color intent with mask holes and edge artifacting.

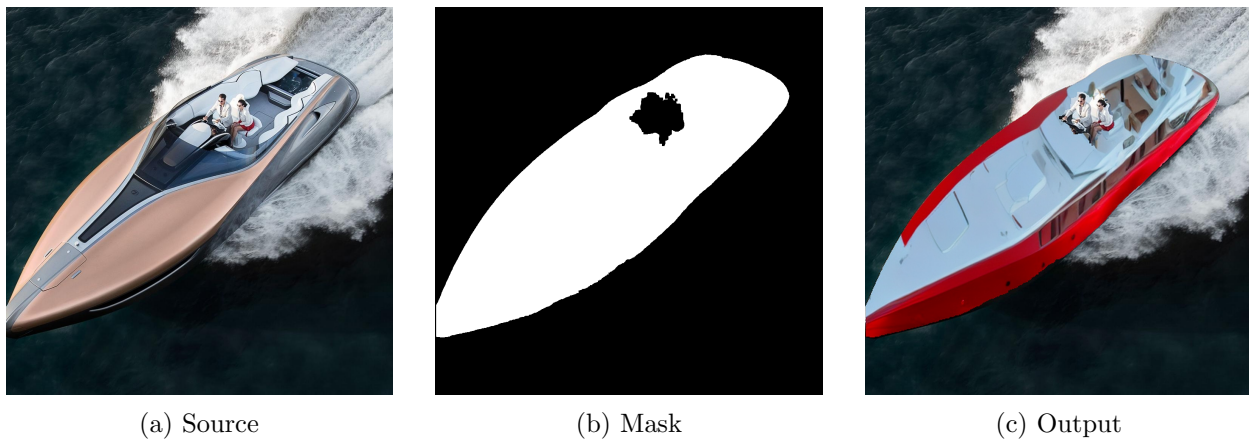


Figure 4: **st_1098**: “Change the luxury boat color to red.” The mask correctly excludes the passengers. The boat shape and color are broadly correct, but the interior is deformed.

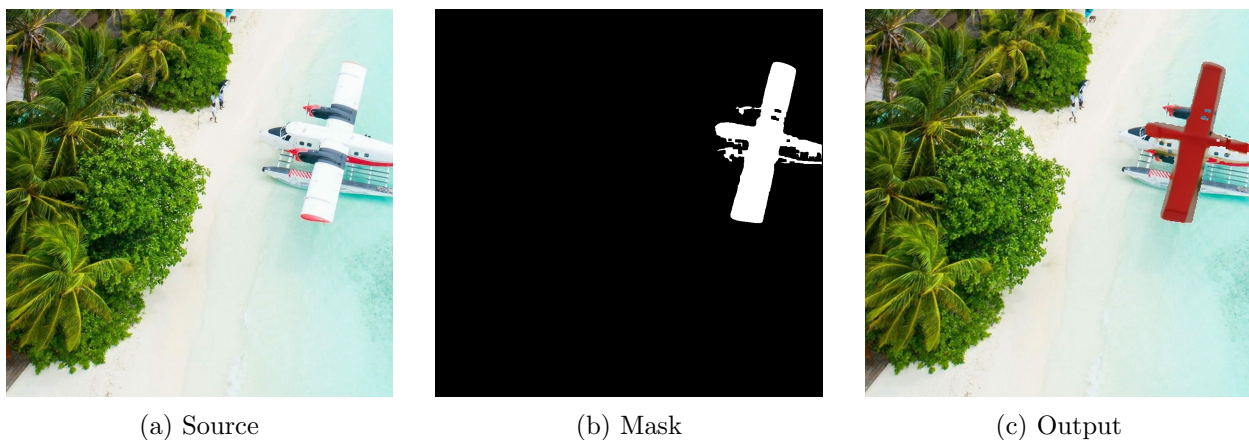
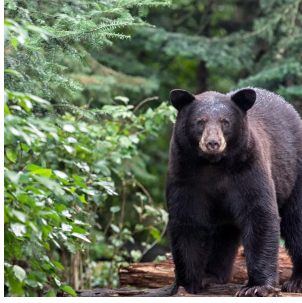
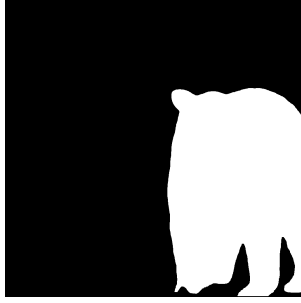


Figure 5: **st_1099**: “Change the seaplane’s color to red.” The mask captures the fuselage and wings but fails around the engine and propellers. The output repaints large surfaces correctly but cannot reconstruct fine detail.



(a) st_1073 Source



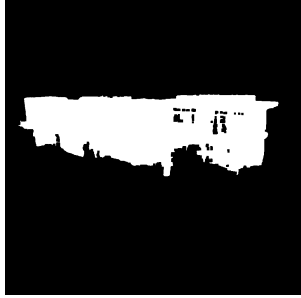
(b) st_1073 Mask



(c) st_1073 Output



(d) st_1106 Source



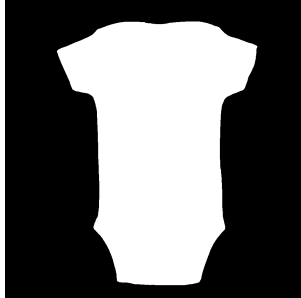
(e) st_1106 Mask



(f) st_1106 Output



(g) st_1130 Source



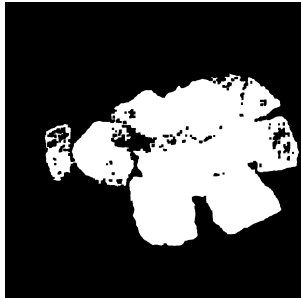
(h) st_1130 Mask



(i) st_1130 Output



(j) st_1135 Source



(k) st_1135 Mask



(l) st_1135 Output

Figure 6: Pipeline failure cases. **st_1073**: “Change the animal’s fur color to a solid shade of brown” - full-surface repaint required; model generates an anatomically incorrect cat like animal. **st_1106**: “Change the building’s facade color to light grey” - architectural subject requires part-level masking the pipeline cannot provide; output is a flat grey fill. **st_1130**: “Change the surface color of the white baby onesie to red” - semantic parsing confusion between source and target color; output is painted white. **st_1135**: “Change the frosting color to a light blue” - no nominal anchor for grounding; mask and diffusion output both degenerate.

Summary of Results

Across the 67-sample benchmark, the pipeline demonstrated that the proposed end-to-end architecture is functional and capable of producing useful edits under a well-defined set of conditions. The results also expose clear boundaries of the current implementation.

The pipeline performs best when the edit target is a single, spatially compact, visually distinct object with smooth contours; the edit instruction specifies a simple attribute change (color or texture) without naming the current state; and the subject noun in the instruction is directly usable as a GroundingDINO query.

Performance degrades predictably when any of the following conditions hold: the target object has complex geometry or fine-grained substructure (machinery, architecture); the mask must cover the entire object, forcing the diffusion model to synthesize rather than recolor (fur, feathers); the edit subject is a semantic part of a larger structure (building facade, frosting on food); or the instruction introduces lexical ambiguity between source and target states.

Three of the four distinct failure modes (over-complex masking, complete-surface repaint, and ambiguous instruction parsing) arise from constraints in the training data scale rather than from architectural defects. With 7,818 training examples across a single edit type, the fine-tuned diffusion model and the Qwen2.5-VL-3B backbone have limited exposure to the long tail of object categories and instruction forms present in the benchmark. The addition of GroundingDINO to the Phase 3 inference pipeline was a practical mitigation for the VLM’s localization limitations on unseen images—the VLM bounding-box predictions, which were accurate on images seen during training, proved insufficiently precise on novel benchmark images.

These results confirm the primary objective of the project: to demonstrate that a modular pipeline combining a fine-tuned VLM, an open-vocabulary grounding model, a promptable segmentation model, and a fine-tuned diffusion inpainting model can be assembled and trained end-to-end within the constraints of a Google Colab. The output quality reflects the training budget, not the ceiling of the architecture.

5 Discussion and Conclusions

Limitations

Several known limitations bound the scope and performance of this pipeline. Fine-tuning Qwen2.5-VL-3B on 7,818 image-instruction pairs over three epochs is insufficient for the model to reliably learn joint JSON format adherence, coordinate space mapping, and spatially-grounded edit localization; bbox outputs were inconsistent enough in practice that the component was replaced with a dedicated grounding model. Because the VLM did not fully learn spatial grounding during training, the last-layer hidden states passed to the diffusion model carry weaker edit-localized context than the architecture intended, retaining useful joint image-and-instruction representations from pretraining but with an attenuated spatially-specific conditioning signal that reduces the expected marginal gain over CLIP text embeddings alone. Previous literature on fine-tuning Qwen2.5-VL-3B, specifically designed for Graphical User Interface grounding tasks, states “...data curation and redundancy reduction strategies, [demonstrate] that randomly sampling a smaller subset with reduced redundancy achieves performance comparable to larger datasets, emphasizing data diversity over sheer volume” and achieved “exceptional accuracy” using only 24k examples (Hsieh and Wei 2025). An other paper by Zhang et al. 2025 finetunes Qwen2.5-VL-3B on 50k examples with the goal “to transform table images into semi-structured representations as such as HTML or Markdown”. The previous literature on fine-tuning Qwen2.5-VL-3B suggests that a high-quality, low redundancy, random sample, data set of around 24k to 50k examples is sufficient to fine-tune the model for specific tasks. Our fine-tuning dataset lacks diversity and volume, proving it to be the primary limitation in this illustration.

Compounding this, the Phase 2 bridge was trained on hidden states extracted using prompts that included full ground-truth segmentation region annotations, which are unavailable at inference time; while the VLM and hidden-state extraction are internally consistent at inference, the distribution shift between training and inference hidden states may further reduce the effectiveness of the VLM conditioning signal. Beyond the VLM components, the another primary constraint on output quality is mask accuracy: when GroundingDINO fails to localize the target object, SAM2 receives a poor bounding box prompt and produces an inaccurate mask, causing the inpainting model to edit pixels outside the intended region regardless of how well it was trained. Even when a suitable mask is generated, due to the models only being trained on 7,818 image edit pairs, the diffusion model has only learned to paint certain objects and lacks breadth to paint more complex objects like a building with windows and doors, animals, and food.

Finally, two resource constraints limit generalizability: training and evaluation are restricted to the “adjust” edit type due to sequential first-N dataset sampling, leaving removal, addition, replacement, and other edit categories untested; and the use of Qwen2.5-VL-3B as the VLM backbone, chosen for its memory footprint, foregoes the stronger spatial reasoning and richer hidden-state representations that larger variants in the Qwen2.5-VL family (7B–72B) would provide. Using a larger parameter VLM, increasing the training samples and diversity, and training on more epochs could result in more consistent/accurate bbox outputs on unseen images, enabling us to forgo using a grounding model and solely rely on the VLM’s spatial reasoning to identify the object and its location.

Future Directions

A natural extension of this project is to grow the training set from 7,818 pairs to roughly 24k–50k while preserving diversity across the 10 edit types and remaining robust to Google Drive’s rate limit, which in practice caps a single download session at around 10k pairs. The challenge is that the source corpus is distributed across many large `.tar.split` shards, and the obvious approach – download shards opportunistically and filter for the examples you want – both wastes bandwidth and biases the sample toward whatever shard order the downloader happens to use.

The key enabler is the per-edit-type Parquet metadata files published alongside the corpus, which list every example’s image ID, edit type, prompt, and containing shard. Downloading all 10 Parquets is cheap (around 100 MB total) and gives a complete population to sample from in memory, without ever opening a tar. Sampling is then straightforward: stratify by edit type, draw $N_k \cdot (1 + \alpha)$ rows per stratum uniformly without replacement, where $\alpha \approx 0.2\text{--}0.5$ is an oversampling buffer that absorbs mask defects discovered only at decode time. Because the full sample is fixed up front, splitting the subsequent download across sessions cannot bias the result – whether retrieved in one chunk or ten, the on-disk set is the same uniform stratified draw – and growing N later uses a seeded re-sample that downloads only the new examples.

The remaining cost is materialising the chosen examples from the tars. Tar has no native random-access index, so a uniform sample spread across hundreds of shards will, in expectation, require touching nearly all of them. Three levers manage this. The simplest is to *bucket the sample by shard* and do a single sequential pass per shard, extracting only the chosen files and discarding the rest – one read per shard rather than per file. A more efficient variant builds a one-time per-shard offset index from tar headers, after which individual examples can be fetched by range read and image bytes for unchosen examples are never transferred at all. The most aggressive option is *cluster sampling*: draw a subset of shards per edit type, then sample examples within those shards. Because shards within an edit type are essentially exchangeable, the result remains an unbiased uniform sample of the corpus while bounding the number of shards ever opened.

In all three variants, downloads are chunked to stay under Drive’s quotas, checkpointed through a per-example status log, and written to a content-addressable layout (e.g. `pairs/{image_id}/`) so that sessions concatenate without renaming or de-duplication. The overall pipeline is therefore: download Parquets, sample once in memory, fetch in chunks. Compared to the current implementation, the benefits are unbiased uniform sampling regardless of how the download is split, clean recovery from quota interruptions, and a single reproducible source of truth for all downstream training and evaluation splits.

Future directions for this work therefore center on smarter data mining and sampling strategies within the constraints of an affordable compute platform. The downside to this implementation is that compared to our original sampling method this would take days to weeks to produce a fine-tuning data set of the size and diversity that we need. Our time and monetary constraints in this project did not allow us to implement this strategy. If we had different hardware that enabled us to keep the tars in persistent memory, this process could be streamlined down to a day of work. We would likely need around 2 to 4 TB of persistent storage, where a local workstation of that capacity could cost upward of \$3000. Under realistic budget constraints, Google Colab and Google Drive remained the most accessible, and the practical gains therefore have to come from methodological refinements to how sampling and download are orchestrated on that platform rather than from infrastructure upgrades.

6 Design Statement

This design statement follows the framework described in Moffat 2025, where the same principles were followed to ensure responsible use of AI tools in the development of this project.

With the rapid evolution of consumer-grade hardware, like GPUs in laptops, larger RAM capacities, and dedicated neural-processing accelerators, individuals will soon be able to train and fine-tune multimodal AI pipelines entirely from their homes in minimal time. In this landscape, generative AI truly shines: instead of poring over dense documentation, learners can pose questions directly (“Why are my VLM hidden states retaining image tokens before mean pooling?” “How do I configure the 9-channel UNet input for Stable Diffusion inpainting?”) and receive immediate, context-aware guidance.

Our collaboration with ChatGPT 5.5, Claude Sonnet 4.6, and Claude Opus 4.7 exemplifies this new workflow. Instead of wrestling with API documentation or scouring forums, we could ask contextual questions—about LoRA module targeting, RLE mask decoding, SAM2 integration, or conditioning tensor concatenation—and receive immediate, tailored guidance. In fact, “knowledge workers spent about a fifth of their time, or one day each work week, searching for and gathering information. If generative AI could take on such tasks, increasing the efficiency and effectiveness of the workers doing them, the benefits would be huge” (Chui et al. 2023).

Imagine a world where anyone, regardless of formal training, can chain together vision-language models, segmentation networks, and diffusion models for local challenges—from automated image restoration to personalized content generation—without wrestling with framework incompatibilities or platform idiosyncrasies. As one Harvard study observed, “AI tools hold significant promise for empowering resource-constrained groups, enabling them to scale operations and enhance communications with limited time and resources” (Chenoweth 2025).

Yet, widespread access also carries responsibility. When guidance is followed uncritically, harms can ensue. As one developer ruefully recounted, “When you delegate a decision to a system you don’t understand and don’t check, you’ve surrendered agency. Worse: you’ve surrendered accountability” (Contieri 2025). Educational research echoes this: “Students’ over-reliance on AI dialogue systems ... affects their critical cognitive capabilities including decision-making, critical thinking, and analytical reasoning” (Zhai, Wibowo, and Li 2024). As generative AI lowers technical barriers, it becomes ever more crucial to teach sound problem-solving principles and ethical reasoning alongside tool usage. To ensure responsible AI usage, we employed the **EVERY** framework: **E**valuate the task, **V**erify facts and sources, **E**ngage with the AI through feedback and oversight, **R**evise the output, and understand **Y**our role in making the final decision (AI for Education 2025).

Ultimately, the barriers to entry for programming are lowering. We are constantly getting access to more powerful computers and creating higher-level abstractions, tutorials, packages, and open-source ecosystems that empower people to create and solve problems without needing deep expertise in low-level hardware/software. By combining human intuition with AI-driven support, while maintaining critical oversight and ethical reasoning, we can unlock deep learning’s full promise, making it more accessible, efficient, and impactful across society.

References

- AI for Education (Sept. 2025). *Updated Framework: How to Use AI Responsibly EVERY Time*. <https://www.aiforeducation.io/blog/updated-ai-every-framework>.
- Chenoweth, Dr. Erica (2025). *How AI Can Support Democracy Movements*. <https://ash.harvard.edu/wp-content/uploads/2025/02/How-AI-Can-Support-Democracy-Movements-Final.pdf>. Harvard Kennedy School.
- Chui, Michael et al. (2023). *The Economic Potential of Generative AI: The Next Productivity Frontier*. <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier>.
- Contieri, Maxi (2025). *From Helpful to Harmful: How AI Recommendations Destroyed My OS*. Accessed May 13, 2025. URL: <https://dev.to/mcsee/from-helpful-to-harmful-how-ai-recommendations-destroyed-my-os-2b70>.
- Dettmers, Tim et al. (2023). “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *arXiv preprint arXiv:2305.14314*. URL: <https://arxiv.org/abs/2305.14314>.
- Ho, Jonathan, Ajay Jain, and Pieter Abbeel (2020). “Denoising Diffusion Probabilistic Models”. In: *arXiv preprint arXiv:2006.11239*. DOI: 10.48550/arXiv.2006.11239. URL: <https://arxiv.org/abs/2006.11239>.
- Hsieh, ZongHan and Tzer-Jen Wei (2025). *Qwen-GUI-3B: A Lightweight Vision-Language Model for Cross-Resolution GUI Grounding*. arXiv: 2506.23491v1 [cs.CV]. URL: <https://arxiv.org/html/2506.23491v1>.
- Liu, Shilong et al. (2024). *Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection*. DOI: 10.48550/arXiv.2303.05499. arXiv: 2303.05499 [cs.CV].
- Moffat, Joshua (May 2025). *Image Captioning Using Convolutional Neural Networks and Long Short-Term Memory*. Course paper, STAT 702: Data Mining, San Diego State University.
- Ravi, Nikhila et al. (2024). “SAM 2: Segment Anything in Images and Videos”. In: *arXiv preprint arXiv:2408.00714*. DOI: 10.48550/arXiv.2408.00714. URL: <https://arxiv.org/abs/2408.00714>.
- Rombach, Robin et al. (2022). “High-Resolution Image Synthesis With Latent Diffusion Models”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695.
- Wang, Peng et al. (2024). “Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution”. In: *arXiv preprint arXiv:2409.12191*. DOI: 10.48550/arXiv.2409.12191. URL: <https://arxiv.org/abs/2409.12191>.
- Ye, Yang et al. (2025). “ImgEdit: A Unified Image Editing Dataset and Benchmark”. In: *arXiv preprint arXiv:2505.20275*. DOI: 10.48550/arXiv.2505.20275. URL: <https://arxiv.org/abs/2505.20275>.
- Zhai, Chunpeng, Santoso Wibowo, and Lily D. Li (2024). “The effects of over-reliance on AI dialogue systems on students’ cognitive abilities: a systematic review”. In: *Smart Learning Environments*. URL: <https://slejournal.springeropen.com/articles/10.1186/s40561-024-00316-7>.
- Zhang, Junyuan et al. (2025). *TRivia: Self-supervised Fine-tuning of Vision-Language Models for Table Recognition*. arXiv: 2512.01248 [cs.CV]. URL: <https://arxiv.org/abs/2512.01248>.